Сопроводительная документация проекта «ФСП Сигнал»

Функциональная спецификация и руководство по эксплуатации

Команда проекта «ФСП Сигнал»

10 октября 2026 года

Содержание

1 НАЗНАЧЕНИЕ РЕШЕНИЯ И СОСТАВ ДОКУМЕНТАЦИИ....3
2 ФУНКЦИОНАЛЬНАЯ И КОМПОНЕНТНАЯ АРХИТЕКТУРА....4
3 МЕХАНИКА ТЕСТИРОВАНИЯ И КАТЕГОРИЗАЦИИ КАНДИДАТОВ....6
4 МЕХАНИКА ОБРАТНОГО ПОДБОРА И ОБЪЯСНЕНИЕ ВЫДАЧИ....8
5 ПРОФИЛЬ, СОГЛАСИЯ, ПРИГЛАШЕНИЯ И КОНТАКТЫ....10
6 ПРОЦЕДУРА ВАЛИДАЦИИ И РЕЗУЛЬТАТЫ ПРОВЕРКИ....12
7 ПРЕДПОЛАГАЕМАЯ ИНТЕГРАЦИЯ С РЕЕСТРОМ ФСП....15
8 ОПИСАНИЕ HTTP API....17
9 БИБЛИОТЕКИ И КОМПОНЕНТЫ С ВЕРСИЯМИ....24
10 СБОРКА, РАЗВЁРТЫВАНИЕ И ЛОКАЛЬНЫЙ ЗАПУСК....26
11 СЦЕНАРИЙ ДЕМОНСТРАЦИИ И ДАЛЬНЕЙШЕЕ РАЗВИТИЕ....30

Документ содержит одиннадцать разделов и охватывает все восемь обязательных тем сопроводительной документации по ТЗ. Описание API и версии компонентов включены в документ; источники контрактов и команды проверки указаны для воспроизведения.

1 Назначение решения и состав документации

«ФСП Сигнал» — веб-платформа обратного подбора ИТ-специалистов. Кандидат заполняет профиль, проходит опрос и тест, подтверждает категорию и разрешает публикацию. Работодатель описывает потребность, получает объяснимую подборку и отправляет адресное предложение с зарплатой. Принятие открывает контакты этой компании; кандидат может отказаться или отозвать доступ.

Категория определяется серверной проверкой; достижения ФСП дополняют оценку. Отсутствие истории ФСП не закрывает основной путь. Контакт инициирует работодатель; опубликованная вакансия не требуется.

Описано объединённое решение A/B/C/D: локальная ветка D, исходный код d30e9c2 от 10 октября 2026 года. На этой версии выполнены проверки; документация добавляется отдельно. Репозиторий: https://github.com/CbICH/aggregator-platform.

Документ соответствует §3.4 ТЗ «Цифровая платформа-агрегатор ИТ-вакансий с верифицированным профилем достижений участника Федерации спортивного программирования», 2026 год.

Обязательный пункт ТЗ	Раздел настоящего документа
Функциональная и компонентная архитектура	2; пользовательские правила — 5
Механика тестирования и устойчивость к распространению заданий	3
Механика подбора и логика категоризации	3 и 4
Собственная процедура валидации и её результаты	6
Схема предполагаемой интеграции с реестром ФСП	7
Описание API	8
Библиотеки и компоненты с версиями	9
Пошаговая сборка, развёртывание и локальный запуск	10

PDF предназначен для сдачи, DOCX — для редактирования. По §5.2 ТЗ отдельно предоставляются презентация и ссылка на репозиторий. Доступный прототип крайне желателен; при его отсутствии нужны видео и локальная инструкция. Инструкция и сценарий включены в разделы 10–11.

Границы описываемой версии

Реализованы регистрация по email, два кабинета, ручной профиль и личный PDF, согласия и видимость данных, опрос, параметризуемое тестирование, подтверждённая категория, компания, потребности, поиск и история подборок, адресные приглашения, решения и отзыв контактов. ФСП представлен явно обозначенным синтетическим реестром. Рабочей интеграции с внешним реестром ФСП и Keycloak в этой версии нет.

Загрузка PDF-резюме с автоматическим распознаванием, самостоятельные отклики на опубликованные вакансии, регулярные задания работодателей, ATS и проверка добросовестности компаний не реализованы. Ручное заполнение профиля предусмотрено ТЗ как допустимый вариант; остальные перечисленные возможности относятся к дополнительному функционалу или развитию за рамками MVP.

2 Функциональная и компонентная архитектура

Компоненты и развёртывание

Система состоит из браузерного приложения, HTTP API, базы данных и SMTP-сервиса, запускаемых отдельными контейнерами. Nginx отдаёт интерфейс и проксирует /api в FastAPI. Браузер работает с общим origin для интерфейса и API.

Компонент	Назначение
React, TypeScript, Vite	Два личных кабинета, формы, таймер теста, подбор, приглашения
Nginx	Статические файлы, маршрутизация SPA, проксирование API
FastAPI	Авторизация, бизнес-правила, DTO, OpenAPI, генерация личного PDF
SQLAlchemy и PostgreSQL	Транзакции, профили, попытки, категории, согласия, компании, предложения
Alembic	Версионирование и применение схемы БД
Mailpit	Приём и просмотр локальных писем подтверждения
DemoFspRegistry	Синтетические достижения и проверка принадлежности демонстрационного ID

Путь запроса: браузер → Nginx → FastAPI → модуль → SQLAlchemy Session → PostgreSQL. SMTP вызывается при регистрации и повторной отправке. PDF формируется сервером; ФСП читается через отдельный порт. Отказ реестра не останавливает подбор по тесту.

Backend — модульный монолит: связанные изменения завершаются одной транзакцией. Для MVP не нужны очереди и распределённые транзакции. Контракты позволяют впоследствии заменить провайдера ФСП или выделить тестирование в отдельный сервис.

Границы модулей и владение данными

Модуль	Ответственность и основные данные
A: core и auth	Пользователи, email, пароли, сессии, токены подтверждения, общая безопасность, ошибки
B: assessment	Опросы, шаблоны, попытки и ответы, оценивание, навыки, категории и история
C: matching и fsp	Компании, потребности, объяснимый подбор, история поиска, связь с ФСП
D: profiles и invitations	Профиль, контакты, согласия, публичная проекция, PDF, приглашения и разрешения на контакты

Идентификатор кандидата совпадает с UUID его учётной записи. Идентификатор компании сервер определяет по текущему работодателю. Клиент не может назначить себе чужую компанию или кандидата. Проверка владельца выполняется в API при каждом обращении к объекту.

Read-only порты AssessmentRead, ProfileRead, CompanyRead и FspRead отдают категорию и навыки, публичный профиль, компанию и достижения. ContactAccess проверяет разрешение конкретной компании. В приложении подключены реальные

данные текущего запроса; tests/doubles.py используется только в изолированных тестах.

Основные сущности и связи

Пользователь имеет роль candidate/employer. Кандидату принадлежат профиль, контакты, настройки, согласия, опросы и попытки. Категория хранит специализацию, грейд, балл, ID и версию оценки, даты назначения и следующей смены; навыки связаны с оценкой.

Компания принадлежит работодателю, потребность — компании. В потребности заданы специализация, грейды, задачи, стек, навыки с весами и формат работы. Поиск хранит компанию, фильтры, копию потребности и UUID участников исходной подборки.

Приглашение связывает компанию и кандидата, хранит снимки условий и оснований, версию, статус и события. Разрешение на контакты хранится отдельно для пары «кандидат — компания» и может быть отозвано после принятия.

Данные имеют UUID и временные метки. Ограничения БД защищают связи и уникальность активных попыток и приглашений. Текущий head — 0013_fsp_searches; новые миграции добавляются поверх истории.

Авторизация и защита запросов

Пароли хешируются Argon2id. Сессионный токен содержит 256 бит случайности; в БД хранится его SHA-256, а в браузере — HttpOnly cookie signal_session. По умолчанию сессия действует восемь часов. В production cookie имеет Secure; SameSite установлен в Lax. Токены не сохраняются в localStorage.

Изменяющий запрос авторизованного пользователя передаёт X-CSRF-Token со значением cookie signal_csrf. Сервер проверяет хеш этого значения и принадлежность сессии. Дополнительно проверяется Origin, в том числе для регистрации и входа. Подтверждённый email и роль проверяются серверными зависимостями.

Одноразовый токен подтверждения email действует 30 минут. Ссылка передаёт токен во фрагменте URL; интерфейс отправляет его в теле запроса подтверждения. Применение защищено условным обновлением БД. Повторная отправка не делает прежнюю непогашенную ссылку недействительной; успешное подтверждение погашает оставшиеся токены пользователя.

Для действий авторизации действуют ограничения по IP и email. По умолчанию лимиты соответствующего действия составляют 30 запросов в минуту по IP и пять за пять минут по адресу; подтверждение ограничивается по IP. При внешнем размещении необходима согласованная настройка доверенного прокси, чтобы лимит учитывал реальный адрес клиента.

Ошибки имеют единый JSON-формат и request_id. Приватные ответы выдаются с запретом кеширования. Наличие чужого объекта не раскрывается через маршруты владельца: возвращается 404. Состояние готовности проверяет доступность нужной схемы БД; несоответствующий стенд возвращает 503.

3 Механика тестирования и категоризации кандидатов

Опрос и выбор уровня

Перед обычным тестированием кандидат проходит опрос: отрасль, специализация, заявленный грейд, роли и опыт в месяцах. Опрос фиксирует исходные сведения и версию справочника. Выбор junior, middle или senior означает уровень предстоящего теста. Он сам по себе не создаёт подтверждённую категорию.

В MVP доступны backend и QA, по три грейда для каждой специализации. Таким образом, имеется шесть категорий. Для каждой подготовлены три семейства заданий; всего используются 18 серверных шаблонов версии v1.

Специализация	Семейство задания	Вес в итоговом балле
Backend	Поведение API и повторные запросы	40
Backend	Данные, SQL и границы	35
Backend	Решение при ограничениях	25
QA	Поиск дефектов	40
QA	Набор проверок и границы	35
QA	Приоритизация рисков	25

Задания содержат структурированные ответы: число, один вариант или набор вариантов. Проверка выполняется сервером по заранее определённым правилам equals и set_equals. За отдельные правила начисляются баллы; итог задания не превышает его вес. Свободный программный код и ответы на естественном языке автоматически не оцениваются. Нейросеть не участвует в выставлении грейда.

Генерация сопоставимых вариантов

При запуске сервер создаёт закрытый случайный seed, выбирает три шаблона и сохраняет конкретные условия и рубрики в БД. Между вариантами меняются числовые параметры, детали ситуации, порядок вариантов и непрозрачные идентификаторы вариантов ответа. Идентификатор шаблона имеет вид backend-middle-api-v1.

Все кандидаты одной категории проверяются по тем же семействам, весам и правилам, но получают индивидуальную форму. Повторное открытие попытки возвращает её сохранённые условия и ответы, а не генерирует новую форму. Публичный DTO содержит условия, вопросы и собственные черновики, но не seed, эталон и рубрику.

Передача готового JSON с ответами не обеспечивает прохождения другой формы: числовые результаты зависят от параметров, а идентификаторы вариантов привязаны к конкретному варианту. Это проверяемая защита от механического копирования ответа. Распространение общего способа решения, помощь другого человека и использование внешних инструментов данным подходом не исключаются. Прокторинг и выявление плагиата не реализованы.

Вариативность сама по себе не доказывает одинаковую сложность. В текущих шаблонах различие грейдов включает параметры и контекст, но психометрическая сопоставимость и соответствие профессиональным уровням требуют отдельной

проверки на людях. Результаты такой проверки нельзя заменять успешной работой генератора.

Попытка, таймер и повторное прохождение

Попытка действует 25 минут по серверному deadline_at. Интерфейс показывает оставшееся время; обновление страницы не продлевает срок. Кандидат сохраняет черновик и продолжает работу в пределах исходного срока. По истечении попытка получает статус expired.

Для одной специализации и целевого грейда допускается одна активная попытка. Число запусков ограничено тремя за 24 часа. Повтор неудачной попытки на тот же уровень доступен через 24 часа. Повторная отправка уже оценённой попытки возвращает сохранённый результат, не создавая новую оценку.

Статусы попытки: in_progress, passed, failed, expired. История отображает только собственные попытки. Недоступная или чужая попытка возвращает 404; преждевременный повтор и нарушение состояния — 409.

Оценка и подтверждение категории

Итог T равен сумме баллов трёх заданий и лежит в диапазоне 0–100. Прохождение требует одновременно $T \geq 70$ и не менее половины максимума в каждом из первых двух, критических семейств. Для весов 40 и 35 минимумы составляют 20 и 17,5 балла. Высокий результат третьего задания не компенсирует провал критического навыка.

При $T \geq 85$ и выполненных критических условиях система рекомендует попробовать следующий грейд. Рекомендация не повышает уровень автоматически. После успешной попытки кандидат явно подтверждает категорию, пока результат актуален; срок подтверждения — семь дней.

Категория имеет код specialization:verified_grade, например backend:middle. Фиксируются assessment_id, assessment_version, балл, навыки и даты. Баллы семейств нормализуются до 0–100. У backend навык api.behavior связан с api и decision: В сохраняет обе записи, а подбор использует последнюю прочитанную запись для этого ID. Отдельное правило объединения таких доказательств не реализовано; его следует закрепить при развитии модели. Связанные навыки не являются независимыми измерениями.

Интервал смены категории — 90 дней. История сохраняет прежние назначения. Неудачная попытка не понижает уже подтверждённый грейд. Повторное подтверждение той же попытки не создаёт новую запись истории. Для грейда ниже или выше требуется успешная соответствующая попытка и соблюдение срока смены.

Регулярные задания

Предложение коротких задач работодателем и обновление актуальности профиля по их результатам предусмотрены ТЗ как желательный функционал. В текущем MVP этот цикл отсутствует. Обычная аттестация и исследовательский пилот A/B реализованы отдельно и не имитируют регулярные задания.

4 Механика обратного подбора и объяснение выдачи

Компания, потребность и допустимые кандидаты

Работодатель создаёт профиль компании с описанием, отраслью, городом и контактным лицом. Затем формулирует потребность: задачи, специализация, один или несколько допустимых грейдов, стек, проверяемые навыки с весами от одного до пяти, формат работы и, при необходимости, диапазон зарплаты.

Подбор читает текущие опубликованные профили D и подтверждённые категории B. Самоописание резюме и заявленный грейд не заменяют результат теста. До расчёта оценки применяются ограничения потребности по специализации и грейду, а также выбранные фильтры.

Для появления кандидата в выдаче необходимы подтверждённый email, заполненный профиль, действующее согласие на обработку, включённая публикация с согласием и подтверждённая категория. Контактные данные, ФИО и свободный текст резюме в выдачу не передаются.

Формула ранжирования

T — балл подтверждённого теста. F — сигнал ФСП по лучшему подтверждённому достижению. R — средневзвешенное соответствие проверенных навыков требованиям потребности. Все компоненты имеют шкалу 0–100.

```
Без выбранной потребности:
S = 0,90 × T + 0,10 × F

С выбранной потребностью:
S = 0,65 × T + 0,25 × R + 0,10 × F

R = Σ(wi × si) / Σwi
```

wi — вес требуемого навыка, si — его подтверждённый балл; если навык не проверен, si = 0. В результате явно указывается «навык не проверялся», а не делается вывод об отсутствии способности. При отсутствии истории ФСП или недоступности реестра F = 0, остальные компоненты сохраняются.

Сигнал ФСП: участник — 30, финалист — 60, призёр или призовое место — 85, победитель — 100; берётся максимальное значение. Достижения даются серверным провайдером и не принимаются в качестве доверенного ввода кандидата. В текущем MVP источник имеет пометку demo.

Вклад ФСП ограничен десятью баллами итоговой оценки. Например, без потребности кандидат с T = 90 без истории получает S = 81,0, а кандидат с T = 75 и победой в ФСП — 77,5. Победа не отменяет разницу проверенной квалификации. Ручное изменение опыта или резюме балл S не повышает.

Формула является прозрачной эвристикой MVP. Коэффициенты не обучены на истории найма и не представляются статистически оптимальными. Результат округляется до одного десятичного знака. При равенстве сортировка стабилизируется UUID кандидата, что исключает случайное перемещение карточек.

Объяснения и фильтры

Карточка показывает подтверждённую специализацию и грейд, общий балл, компоненты test/relevance/fsp и основания: подтверждение категории, проверенные навыки, непроверенные требования, достижения, отсутствие истории или недоступность реестра. Работодатель может оценить не только позицию карточки, но и основание её попадания в подборку.

Доступны фильтры специализации, грейда, кода категории, публичного стека и наличия достижений ФСП. Стек потребности используется при отсутствии явно заданного стека фильтра. Скрытый кандидатом стек не участвует в фильтрации, чтобы состав и размер выдачи не раскрывали скрытые значения.

Пагинация подборки использует limit от 1 до 50 и offset; ответ содержит total и next_offset. Публичный банк D использует UUID-курсор after. Эти два способа не взаимозаменяемы.

История подборок и переход к предложению

Сохранённый поиск принадлежит компании. В БД записываются фильтры, копия потребности и UUID участников исходной подборки. Частные профили и контакты в снимок не копируются.

При повторном открытии система заново читает текущие разрешённые данные, категорию и достижения. Кандидат, скрывший профиль или отозвавший обработку, исчезает и из старой подборки. Ответ содержит исходное количество и число недоступных профилей; оценки оставшихся кандидатов пересчитываются. История поиска сохраняет контекст отбора, но не является архивом неизменных оценок.

Из карточки работодатель переходит к приглашению. Интерфейс переносит контекст собственной потребности и доступные условия в форму; зарплату в предложении необходимо указать обязательно. Сервер повторно проверяет владельца потребности, публикацию кандидата и допустимость действия. Сохранённая карточка не предоставляет постоянного права связаться со скрытым кандидатом.

5 Профиль, согласия, приглашения и контакты

Личный профиль кандидата

Кандидат вручную заполняет ФИО, роли, опыт, стек, мягкие навыки, текст резюме и контакты. Указанные сведения являются самоописанием; тестовая категория хранится отдельно. Публично используется псевдоним, а не ФИО.

Первое сохранение требует согласия на обработку профиля по актуальной версии текста. Последующие обновления передают expected_version. Если другую версию уже сохранили в соседней вкладке, сервер возвращает 409; частичное изменение и молчаливое перетирание данных не допускаются. Пользователь обновляет форму и повторяет действие.

Публичная проекция может содержать роли, стек и стаж, если соответствующие поля разрешены. Скрытое поле возвращается как null и отмечается в hidden_fields. Свободный текст резюме, мягкие навыки, ФИО и контакты не передаются в карточку работодателя. Публикуемый стек ограничен допустимыми техническими тегами.

Согласия и отзыв

Разделены три действия: обработка профиля, публикация данных и передача контактов. Согласие при регистрации не включает автоматически публикацию и передачу. Для каждого действия сохраняются тип, версия текста, время принятия и время отзыва.

Отключение публикации немедленно исключает профиль из новых выдач и повторного открытия старых подборок. Уже отправленные приглашения остаются доступными их сторонам вместе со снимком условий. Отключение публикации само по себе не равно отзыву всех ранее переданных контактов; для этого предусмотрено отдельное действие.

Отзыв передачи контактов закрывает действующие разрешения. Повторное чтение принятого приглашения и повторная отправка прежнего решения не восстанавливают отозванный доступ. Для новой передачи требуется новое допустимое действие с явным согласием.

Отзыв обработки выключает публикацию и закрывает разрешения на контакты; выгрузка профиля в PDF становится недоступной. Существующие необходимые записи взаимодействия не объявляются физически удалёнными. Автоматическая полная ликвидация аккаунта и утверждённый юридический срок хранения в MVP не реализованы.

Личный PDF

Собственный PDF доступен подтверждённому кандидату с действующей обработкой профиля. Он содержит личные сведения, контакты, подтверждённую категорию и доступную информацию ФСП. Другой пользователь не может скачать этот документ через API.

ReportLab формирует PDF на сервере. Включённый шрифт Noto Sans обеспечивает кириллицу. Пользовательский текст экранируется, переносы и длинные значения обрабатываются при вёрстке. Ответ имеет тип application/pdf и не

кешируется как публичный ресурс. Этот PDF профиля отличается от настоящей сопроводительной документации проекта.

Создание адресного приглашения

Компания указывает кандидата, описание предложения, способ связи и диапазон зарплаты. Валюта — RUB, период — месяц, tax_basis — gross, то есть до удержания налогов. Обе границы — положительные целые числа, минимум не выше максимума. В потребности зарплата может отсутствовать, в приглашении — обязательна.

Связь с собственной потребностью необязательна. Привязка к опубликованной вакансии в обязательном сценарии отсутствует. Если клиент передаёт неподдерживаемую vacancy_id, сервер не создаёт фиктивную вакансию.

Сохраняются условия, сведения компании и основания на момент отправки. Дальнейшее редактирование компании или категории кандидата не переписывает старое предложение. Контакт работодателя виден кандидату до принятия.

Ключ Idempotency-Key длиной 8–128 символов защищает повторную отправку. Тот же ключ с тем же содержимым возвращает прежнее приглашение; с другим содержимым — 409. Ограничение активного приглашения и лимит 30 новых приглашений в час защищают от дублей и массовой отправки.

Статусы и решения

```
sent → viewed → accepted
              → declined

sent → accepted / declined также допустимы
```

Чтение приглашения не меняет статус. Просмотр фиксируется отдельным POST-запросом. Принятие или отклонение доступно только адресату. Финальное решение защищено expected_version, блокировкой профиля и условным обновлением; конкурентные действия не дают двух противоречащих исходов.

При принятии кандидат явно соглашается передать контакты этой компании по актуальной версии текста. Решение, событие, согласие и разрешение фиксируются одной транзакцией. При отклонении разрешение не возникает. Обе стороны видят статус и историю собственных приглашений.

Компания получает контакты через отдельный защищённый маршрут только при действующем разрешении. Другой работодатель не получает доступ даже при совпадающих фильтрах или знании UUID кандидата. После отзыва тот же маршрут вновь закрыт.

6 Процедура валидации и результаты проверки

Назначение и воспроизводимость

Проверка разделена на корректность алгоритмов, целостность данных и прав, сквозной пользовательский сценарий и измерение качества механики на людях. Первые три уровня выполнены локально. Четвёртый подготовлен как отдельный пилот и пока не имеет наблюдений. Такое разделение позволяет оценить работающий прототип без приписывания ему непроведённой экспертной проверки.

Дата технической проверки — 10 октября 2026 года; версия кода — d30e9c2. Данные аккаунтов и достижений синтетические. Тесты API используют временный SQLite либо отдельную тестовую PostgreSQL; производственный volume не передаётся в очистку тестового окружения.

Проверка генератора и распространения ответов

Для каждого из 18 шаблонов проверяются 100 воспроизводимых вариантов. Серверный эталон должен давать полный вес задания; все публичные условия в этой выборке должны отличаться. Дополнительно проверяется отсутствие строк seed/expected в публичной форме. Это техническая проверка утечки известных служебных значений, а не универсальное доказательство отсутствия любой утечки.

В каждой из шести категорий выбрано 20 пар разных вариантов. Эталонные ответы первой формы без изменения передаются второй форме; оценка и критические условия применяются обычной функцией проверки. Собственный скрипт расположен в validation/assessment-template-check.py.

Измерение	Результат
Проверено шаблонов	18 из 18
Проверено вариантов	1 800
Различных публичных форм на шаблон	100 из 100
Балл собственного эталона	Полный вес во всех проверках
Проверок прямого переноса ответов	120
Прохождений после переноса	0 из 120

Отчёт validation/assessment-template-report.json воспроизводится приведённой ниже командой. Повторный запуск при подготовке документа подтвердил те же показатели. Результат относится к прямому копированию структурированных ответов и не измеряет помощь извне, запоминание принципа решения или сопоставимость сложности между грейдами.

```
PYTHONPATH=apps/api .venv/bin/python \
  validation/assessment-template-check.py
```

Проверка подбора и правил категоризации

В контролируемых наборах проверяются оба варианта формулы, вес требуемых навыков, нулевой вклад непроверенного навыка, ограниченный сигнал ФСП и случай отсутствия истории. Категория читается из B, ручной грейд её не подменяет. Дополнительно проверяются фильтры, порядок выдачи, владение потребностями и поисками, исключение скрытых полей и повторное чтение истории после отзыва.

Ожидаемые результаты задаются в API-тестах как независимые условия: чужая компания не читает поиск, скрытый профиль не возвращается, сильный подтверждённый тест не заменяется произвольным резюме, подмена достижения клиентом не принимается. Проверки доступны в test_matching.py, test_fsp.py и test_bcd_integration.py.

Эти результаты подтверждают реализацию заявленных правил на контролируемых данных. Precision@k, согласие реальных работодателей с выдачей и успех найма на внешней размеченной выборке не измерены. Для оценки полезности подбора требуется отдельная ручная разметка пар «потребность — кандидат», независимая от формулы S.

API, миграции и браузер

Проверка	Фактический результат
Полный API-набор на SQLite	149 passed; 0 failures/errors/skips; 39,83 с
Полный API-набор на PostgreSQL	149 passed; 0 failures/errors/skips; 59,99 с
Финальная проверка matching на PostgreSQL	10 passed после уточнения company-owned lookup
Production-сценарии Chromium	4 passed
Изолированный сценарий D	1 passed; управляемые порты B/C
Миграции SQLite	Проверены 6 исходных состояний; данные сохранены
Миграции PostgreSQL	upgrade/check, downgrade к core и повторный upgrade/check
Текущая схема стенда	head 0013_fsp_searches; Alembic не обнаружил изменений
Статические проверки	Ruff, форматирование 86 файлов, TypeScript и production build
Контракты	Повторная генерация не изменила OpenAPI, JSON Schema и API-типы

Полный API-набор включает авторизацию, опрос и тестирование, владение объектами, приватность, PDF, ФСП, подбор, приглашения, идемпотентность и конкурентные решения. Финальный company-owned lookup дополнительно проверен соответствующей группой PostgreSQL-тестов. Эти результаты не обозначаются как новый запуск GitHub CI: коммит проверен локально.

Основной браузерный сценарий использует рабочие модули: регистрация и письмо → профиль → собственный demo-ФСП ID → обычный тест B → подтверждение категории → компания и потребность → live-подбор, фильтры и история → приглашение → принятие → контакты → отзыв → скрытие и отвязка ФСП.

Дополнительный изолированный сценарий D подменяет внешние read-порты управляемыми данными и проверяет два кабинета, две компании, принятие, отказ, приватность и PDF. Он дополняет основной сценарий, но не является доказательством интеграции с реальными B/C.

Интерфейс проверен на ширине 1 280 и 390 пикселей. В просмотренных состояниях не обнаружены горизонтальное переполнение и неожиданные alert. Это проверка адаптации основных экранов, а не полная сертификация всех браузеров и требований доступности.

Проверка времени ответа

На локальном стенде выполнено 120 GET-запросов к публичному банку, ранжированию и сохранённому поиску. Использовались 20 параллельных клиентов одной подтверждённой сессии и семь публичных профилей. Все ответы — HTTP 200; медиана 316,52 мс, 95-й процентиль 431,11 мс, максимум 461,49 мс; длительность серии — около двух секунд.

Измерение показывает работу небольшого MVP-стенда при конкурентном чтении. Оно не моделирует 20 независимых аккаунтов, массовое тестирование или большой банк кандидатов. Для масштабирования необходимо измерять отдельно объём БД, число сессий, запись ответов, SMTP и внешние обращения к реестру.

Пилот сопоставимости и экспертной оценки

Подготовлен собственный протокол для шести–восьми добровольцев. Для одной специализации создаются формы A и B; порядок A/B и B/A балансируется. До первой формы независимый эксперт либо два рецензента фиксируют ручной грейд. Расхождение рецензентов разрешается с записью причины до просмотра автоматического результата.

Участник проходит обе формы с обычным серверным сроком 25 минут. До завершения пары ему не показываются баллы и эталоны. Пилотные попытки не назначают обычную категорию, не публикуются работодателю и не участвуют в подборе.

После ручной оценки и двух завершённых форм оператор запускает закрытый CLI-подсчёт и экспортирует обезличенные результаты. В отчёт включаются даты, число участников, порядок, оценки, время и все причины исключений. Анализ рассчитывает распределения баллов и времени, разницу A/B, согласие решений, соответствие ручной оценке и разведочное разделение групп опыта.

Ориентиры протокола — медианная разница форм не более десяти баллов и согласие не менее 80%. Это целевые критерии проверки гипотезы, а не полученные значения. Ограничения пилота: малая добровольная выборка, обучение между формами, отсутствие контроля внешней помощи и невозможность обобщить результат на весь рынок.

На дату документа участники и независимые рецензенты не привлекались. В validation/assessment-pilot/metadata.json указан статус not_measured. Экспертное соответствие грейдов, устойчивость результатов между людьми и дискриминативность пока не установлены. Подробный операторский протокол находится в docs/assessment-pilot.md; анализ — в validation/assessment-pilot/analyze.py.

7 Предполагаемая интеграция с реестром ФСП

Действующий демонстрационный адаптер

DemoFspRegistry использует фиксированные серверные записи apps/api/app/modules/fsp/demo_registry.json. Для проверенных адресов демонстрационных аккаунтов доступны синтетические истории победителя, финалиста и участника без достижений; обычный подтверждённый аккаунт получает собственный пустой demo-ID. Значения не относятся к реальным участникам ФСП.

Кандидат сначала получает собственный доступный ID, затем связывает именно его. Проверка использует candidate_id и подтверждённый email текущего пользователя. Ввод чужого ID отклоняется. Адрес с уникальным суффиксом +… у демонстрационного аккаунта получает отдельный ID с UUID.

Демонстрационная принадлежность не является подтверждением владения реальной внешней учётной записью. Для внешнего реестра её следует заменить доверенной федеративной авторизацией или предусмотренным владельцем реестра доказательством владения.

Статус	Значение и поведение
not_linked	Связи нет; кандидат продолжает работу по тесту
linked_empty	Связь подтверждена, достижения отсутствуют; F = 0
linked_with_data	Есть серверные достижения; рассчитывается F
unavailable	Реестр недоступен; подбор продолжается с F = 0

Владелец видит собственный ID. Работодатель получает статус, источник и достижения, но не внешний идентификатор. Отвязка работает даже при недоступном реестре. Профиль, PDF и подбор используют один FspRead, чтобы не расходиться в трактовке состояний.

Целевая схема с Keycloak и внешним реестром

В ТЗ указано предполагаемое использование Keycloak для ФСП ID, но API и структура реестра на хакатоне не предоставляются. Следующая схема является проектным предложением. URL, scopes, поля и правила реального реестра необходимо согласовать с ФСП; в текущем коде они не подключены.

1. Зарегистрировать платформу как OIDC-клиент Keycloak и согласовать допустимые redirect URI, scopes и форму согласия.
2. По действию кандидата запустить Authorization Code Flow с PKCE, state и nonce. Привязать запрос к текущей локальной сессии.
3. На сервере проверить подпись, issuer, audience, срок и nonce. Внешнюю личность определять по паре issuer/sub; email сам по себе не считать устойчивым идентификатором.
4. По доверенному идентификатору получить разрешённые достижения из официального реестра через FspRegistry.
5. Нормализовать согласованные поля и выдать FspEvidence с source = registry, датой получения, статусом и доказательствами.

6. Записать связь только для текущего кандидата; запретить произвольный выбор чужого sub и конфликтующее связывание.

7. Передать в подбор и PDF разрешённую проекцию, сохранив приватность внешнего ID. При отзыве отключить дальнейшее чтение и публикацию соответствующих данных.

При интеграции необходимы ограниченные тайм-ауты, повтор с задержкой, ограниченный кеш с датой свежести и журнал технических отказов без токенов. Различаются отсутствие истории, истечение авторизации, отказ провайдера и недоступность сети. Сбой внешней системы не должен блокировать профиль, тестирование или приглашения.

Кеширование, хранение refresh token, перечень достижений, основания обработки и сроки удаления определяются совместно с владельцем реестра. Не следует объявлять эти механизмы реализованными на основании работающего demo-адаптера.

8 Описание HTTP API

Спецификация, адрес и формат

Базовый путь методов — /api/v1. Локальная интерактивная документация: http://localhost:8080/api/docs; актуальная схема: http://localhost:8080/api/openapi.json. Копия контракта находится в contracts/openapi.json, общие DTO — в contracts/schemas.json. TypeScript-типы интерфейса генерируются из этого контракта.

OpenAPI описывает все 57 операций, пути и query-параметры, схемы запросов и ответов, ограничения полей, cookie-авторизацию и коды ошибок. Для полного набора полей вложенного DTO следует открыть components.schemas в указанном файле. Перечень ниже включает обычную аттестацию и отдельный пилот; операторские CLI-команды в HTTP API не включаются.

JSON-запросы передаются с Content-Type: application/json. Для закрытых методов требуется signal_session; для изменяющих закрытых методов дополнительно X-CSRF-Token. Маршруты /candidate и /assessments доступны кандидату, /company и /employer — работодателю. Приглашения доступны только участникам конкретного взаимодействия. Для рабочих кабинетов нужен подтверждённый email.

В таблицах пути указаны относительно /api/v1, слева — метод и назначение, справа — DTO запроса и успешного ответа. «Без тела» означает отсутствие JSON-тела. Число в скобках — успешный HTTP-статус.

Система

Метод, путь и назначение	Запрос → успешный ответ
`GET /health` Проверка процесса	Без тела → Health (200)
`GET /ready` Проверка готовности схемы и БД	Без тела → Health (200)

Авторизация

Метод, путь и назначение	Запрос → успешный ответ
`GET /auth/registration-policy` Текст и версия согласия регистрации	Без тела → RegistrationPolicy (200)
`POST /auth/register` Регистрация	RegisterRequest → Message (202)
`POST /auth/resend-verification` Повторная отправка письма	EmailRequest → Message (202)
`POST /auth/verify-email` Подтверждение email	VerifyRequest → Message (200)
`POST /auth/login` Вход и выдача cookie	Credentials → UserResponse (200)
`GET /me` Текущая учётная запись	Без тела → UserResponse (200)
`POST /auth/logout` Завершение сессии	Без тела → Message (200)

Справочники

Метод, путь и назначение	Запрос → успешный ответ
`GET /catalogs` Справочники категорий и навыков	Без тела → Catalogs (200)

Опрос, тестирование и пилот

Метод, путь и назначение	Запрос → успешный ответ
`POST /candidate/surveys` Сохранение опроса	SurveyCreate → SurveyResponse (201)
`GET /candidate/surveys` История своих опросов	Без тела → SurveyHistory (200)
`GET /candidate/pilot` Состояние участия в пилоте	Без тела → PilotState (200)
`POST /candidate/pilot/join` Вступление по приглашению	PilotJoin → PilotState (201)
`POST /candidate/pilot/attempts` Следующая пилотная форма	Без тела → PilotAttemptPublic (201)
`GET /candidate/pilot/attempts/{attempt_id}` Собственная пилотная форма	Без тела → PilotAttemptPublic (200)
`PUT /candidate/pilot/attempts/{attempt_id}/answers` Черновик пилотной формы	AssessmentAnswerUpdate → PilotAttemptPublic (200)
`POST /candidate/pilot/attempts/{attempt_id}/submit` Завершение пилотной формы	Без тела → PilotAttemptPublic (200)
`POST /assessments` Начало обычной попытки	AssessmentStart → AssessmentResult (201)
`GET /assessments/{assessment_id}` Собственная попытка	Без тела → AssessmentResult (200)
`PUT /assessments/{assessment_id}/answers` Сохранение ответов	AssessmentAnswerUpdate → AssessmentResult (200)
`POST /assessments/{assessment_id}/submit` Серверная оценка	Без тела → AssessmentResult (200)
`GET /candidate/assessments` История обычных попыток	Без тела → AssessmentHistory (200)
`GET /candidate/category` Текущая категория и ограничения	Без тела → CategoryState (200)
`GET /candidate/category/history` История назначений	Без тела → CategoryHistory (200)
`POST /candidate/category/confirm` Явное подтверждение категории	CategoryConfirm → CategoryState (200)

Компания, потребности и подбор

Метод, путь и назначение	Запрос → успешный ответ
`POST /company` Создание своей компании	CompanyCreate → CompanyResponse (201)
`GET /company` Профиль своей компании	Без тела → CompanyResponse (200)
`PATCH /company` Обновление компании	CompanyUpdate → CompanyResponse (200)
`POST /employer/needs` Создание потребности	NeedCreate → NeedResponse (201)
`GET /employer/needs` Список своих потребностей	Без тела → NeedList (200)
`GET /employer/needs/{need_id}` Собственная потребность	Без тела → NeedResponse (200)
`GET /employer/categories` Категории и размер публичного банка	Без тела → CategoryList (200)
`GET /employer/matching/candidates` Актуальный ранжированный подбор	Без тела → CandidateList (200)
`POST /employer/searches` Сохранение подборки	SearchFilters → SearchResponse (201)
`GET /employer/searches` История своей компании	Без тела → SearchHistory (200)
`GET /employer/searches/{search_id}` Повторное открытие поиска	Без тела → SearchResponse (200)

Связь с ФСП

Метод, путь и назначение	Запрос → успешный ответ
`GET /candidate/fsp` Собственная связь и доступный ID	Без тела → FspSelf (200)
`POST /candidate/fsp-link` Связывание собственного ID	FspLinkCommand → FspSelf (200)
`DELETE /candidate/fsp-link` Отвязка ID	Без тела → FspSelf (200)

Профиль и приватность

Метод, путь и назначение	Запрос → успешный ответ
`GET /candidate/profile.pdf` Собственный PDF	Без тела → PDF (200)
`GET /candidate/privacy` Видимость, согласия, их тексты	Без тела → PrivacyResponse (200)
`PATCH /candidate/privacy` Изменение видимости и отзыв доступа	PrivacyUpdate → PrivacyResponse (200)
`POST /candidate/consents` Принятие или отзыв согласия	ConsentCommand → PrivacyResponse (200)
`GET /candidate/profile-policy` Текст обработки профиля	Без тела → ProfileProcessingPolicy (200)
`GET /candidate/profile` Собственный профиль	Без тела → CandidateSelf (200)
`PATCH /candidate/profile` Сохранение собственного профиля	ProfilePatch → CandidateSelf (200)

Приглашения

Метод, путь и назначение	Запрос → успешный ответ
`GET /employer/invitation-context` Контекст формы приглашения	Без тела → EmployerInvitationContext (200)
`GET /employer/candidates` Публичный банк без ранжирования	Без тела → PublicProfileList (200)
`GET /employer/candidates/{candidate_id}` Доступный публичный профиль	Без тела → CandidatePublic (200)

`GET /employer/candidates/{candidate_id}`
`/contacts`
Контакты при действующем разрешении
	Без тела
	→ CandidateContacts (200)

`POST /invitations` Отправка приглашения работодателем	InvitationCreate → InvitationResponse (201)
`GET /invitations` Собственные приглашения текущей стороны	Без тела → InvitationListResponse (200)
`GET /invitations/{invitation_id}` Приглашение, доступное стороне	Без тела → InvitationResponse (200)
`GET /invitations/{invitation_id}/events` История событий	Без тела → InvitationHistory (200)
`POST /invitations/{invitation_id}/view` Фиксация просмотра адресатом	Без тела → InvitationResponse (200)
`POST /invitations/{invitation_id}/decision` Принятие или отказ адресата	InvitationDecision → InvitationResponse (200)

Основные входные данные и ограничения

Регистрация: email; password длиной 12–128 символов; role candidate/employer; consent_accepted = true; consent_version из registration-policy. Версия политики не должна подставляться произвольно. Регистрация и повторная отправка возвращают

202; при отказе SMTP аккаунт сохраняется, а пользователь может повторить отправку после устранения причины.

Опрос: industry, specialization, declared_grade, roles и experience_months. Начало теста: specialization и target_grade. Черновик: answers, словарь «ID задания → словарь ответов». Конкретные имена полей и идентификаторы вариантов берутся только из выданной формы. Подтверждение категории: assessment_id успешной собственной попытки.

Профиль: expected_version и изменяемые поля; при первой записи также согласие обработки и его версия из profile-policy. Приватность: expected_version, visible_fields, publication_enabled и при включении публикации — принятие и версия её согласия. Команда согласия: consent_type, action, version и expected_version.

Компания: название, описание, отрасль, город, контактное лицо и email. Потребность: title, description, specialization, grades, declared_stack, required_skills, work_format, необязательные salary_min/salary_max. Навыки уникальны, берутся из справочника и соответствуют специализации; зарплата задаётся обеими границами или не задаётся вовсе.

Поиск: need_id, specialization, grade, category_code, stack и with_fsp_only. Для списка кандидатов matching используются те же фильтры в query и limit/offset; сохранённый поиск создаётся JSON-запросом. История и конкретный поиск принадлежат текущей компании.

Приглашение: candidate_id, description, salary и company_contact; необязательно need_id и evidence_refs. Ответ содержит снимки условий, статус и версию. Решение: action accept/decline и expected_version; принятие дополнительно требует contact_consent_accepted = true и актуальную contact_consent_version.

Примеры запросов

Начало обычного тестирования после опроса:

```
{
  "specialization": "backend",
  "target_grade": "middle"
}
```

Создание подборки без привязки к потребности:

```
{
  "specialization": "backend",
  "grade": "middle",
  "stack": ["python"],
  "with_fsp_only": false
}
```

Зарплатная часть приглашения:

```
{
  "min": 150000,
  "max": 200000,
  "currency": "RUB",
  "period": "month",
  "tax_basis": "gross"
}
```

Здесь показан вложенный объект salary, а не полное тело InvitationCreate. UUID кандидата и потребности, версия профиля, согласие и ключ идемпотентности должны браться из текущего состояния API.

Успешные ответы и ошибки

AssessmentResult содержит идентификатор, специализацию, уровень, статус, срок и вопросы; после оценки — score, family_scores, passed и next_grade_recommended. CandidateList содержит items, total, applied_need_id, limit и next_offset. SearchResponse дополняет выдачу сведениями сохранённого поиска и актуальной доступностью участников.

CandidateSelf — собственный профиль и версия; CandidatePublic — только разрешённые поля и подтверждённая категория; CandidateContacts — доступные контакты для допущенной компании. InvitationResponse содержит предложение, статус, снимки и версию. Профиль PDF возвращается бинарным application/pdf.

Ошибки представлены единым объектом:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Проверьте введённые данные",
    "fields": null
  },
  "request_id": "идентификатор запроса"
}
```

Это пример формы ошибки; конкретные message и fields зависят от нарушения. Поля ошибки валидации не должны раскрывать пароль или закрытое значение. Для диагностики пользователь передаёт request_id.

HTTP-статус	Значение
400	Некорректный запрос или формат операции
401	Нет действующей сессии
403	Неверная роль, email, CSRF/Origin или недостаточное разрешение
404	Объект отсутствует либо не принадлежит текущей стороне
409	Конфликт версии, состояния, повторов или ключа идемпотентности
422	Нарушены ограничения DTO или бизнес-валидация
429	Превышен лимит запросов
500	Внутренний отказ; детали не раскрываются клиенту
503	Стенд, почтовый сервис или внешний провайдер недоступен

Общий набор ошибок включён в контракт большинства маршрутов. Это не означает, что каждый код возникает в каждом методе. Конкретные условия задаются реализацией: например, FSP_ID_NOT_OWNED — 422, ASSESSMENT_RETRY_COOLDOWN и конфликт expected_version — 409.

9 Библиотеки и компоненты с версиями

Зафиксированное окружение

Версии ниже взяты из Dockerfile, docker-compose.yml, requirements*.txt и package-lock.json описываемой версии. Воспроизводимая сборка использует lock-файлы и npm ci, а не установку последних доступных пакетов. Docker Engine и Compose v2 являются внешними требованиями окружения; точная версия хоста проектом не зафиксирована.

Компонент	Версия
Python, контейнер API	3.12.14
Node.js, сборка frontend	22.12.0
PostgreSQL, образ	17.6-alpine
Nginx, образ	1.28.0-alpine
Mailpit, образ	v1.27.8
Frontend приложения	0.1.0
Шаблоны аттестации	v1; assessment-v1
Миграция схемы	0013_fsp_searches

Backend

Runtime-зависимости и их транзитивные компоненты перечислены полностью. Хеши дистрибутивов находятся в apps/api/requirements.txt; зависимости проверки — в requirements-dev.txt.

Пакет	Версия
alembic	1.20.0
annotated-doc	0.0.5
annotated-types	0.8.0
anyio	4.15.1
argon2-cffi	25.1.0
argon2-cffi-bindings	26.1.0
cffi	2.1.1
charset-normalizer	3.5.2
click	8.5.0
dnspython	2.8.0
email-validator	2.3.0
fastapi	0.142.2
greenlet	3.5.6
h11	0.16.0
idna	3.20
mako	1.4.3
markupsafe	3.0.4
opentelemetry-api	1.45.0
pillow	12.3.0
psycopg	3.3.6
psycopg-binary	3.3.6
pycparser	3.0

Пакет	Версия
pydantic	2.13.5
pydantic-core	2.46.5
pydantic-settings	2.15.0
python-dotenv	1.2.4
reportlab	4.5.1
sqlalchemy	2.0.54
starlette	1.7.0
typing-extensions	4.16.0
typing-inspection	0.4.4
uvicorn	0.54.0

Frontend и проверка

В таблице перечислены прямые зависимости приложения и инструментов из apps/web/package.json. Полный граф транзитивных пакетов с версиями и integrity находится в package-lock.json.

Пакет приложения или проверки	Версия
react	19.3.0
react-dom	19.3.0
react-router-dom	7.18.4
@playwright/test	1.63.0
@types/node	26.6.4
@types/react	19.3.0
@types/react-dom	19.3.0
@vitejs/plugin-react	6.1.2
openapi-typescript	7.13.0
typescript	5.9.3
vite	8.3.2
pytest	9.1.1
httpx	0.28.1
ruff	0.16.10
aiosmtpd	1.4.6
pip-tools	7.6.1
pypdf	6.19.0

Для API-проверок используются pytest, HTTPX и Ruff; для браузера — Playwright Chromium. Их версии фиксируются в соответствующих lock-файлах. При изменении зависимости требуется обновить lock-файл, повторить проверки и актуализировать документ.

Для личного PDF включён Noto Sans с файлом лицензии OFL-1.1 в assets модуля profiles. Times New Roman, Arial и Courier New используются только при оформлении настоящего документа по образцу; в приложение эти системные шрифты не добавлялись. Синтетический реестр и данные проверки не содержат реальных достижений участников ФСП.

10 Сборка, развёртывание и локальный запуск

Подготовка окружения

Нужны Git, Python 3 для запуска scripts/dev.py, работающий Docker Engine или Docker Desktop и Docker Compose v2. Для разработки вне контейнеров нужны Python 3.12 и Node.js 22.12 или совместимая версия, поддерживаемая Vite. Для macOS можно использовать Docker-совместимый Colima.

Команды выполняются из корня вложенного репозитория aggregator-platform. Файл .env создаётся локально и не добавляется в Git. В примере используются только демонстрационные реквизиты; перед публичным размещением их необходимо заменить.

1. Получить исходники и перейти в каталог проекта.
2. Проверить доступность Docker и Compose.
3. Скопировать .env.example в .env.
4. Запустить сборку и дождаться готовности API и базы.

```
git clone https://github.com/CbICH/aggregator-platform.git
cd aggregator-platform
docker version
docker compose version
cp .env.example .env
python3 scripts/dev.py
```

Для воспроизведения описанной версии нужен исходный коммит d30e9c2 или опубликованная командой ветка, содержащая его. Наличие этого локального коммита на GitHub не предполагается автоматически. После публикации сдачи следует зафиксировать адрес соответствующего коммита.

Скрипт выполняет docker compose up -d --build --wait. Если рабочий путь содержит кириллицу, исходники для сборки временно копируются в ASCII-путь; локальные секреты и node_modules туда не включаются. В каталоге без такой особенности можно вызвать Docker Compose напрямую.

```
docker compose up -d --build --wait
docker compose ps
docker compose logs --tail 100 api
```

При старте API применяет alembic upgrade head. PostgreSQL сохраняется в именованном volume. Обычный docker compose down останавливает стенд и не удаляет базу; параметр -v удаляет volume и не используется для обычного завершения работы.

Адреса и проверка готовности

Ресурс	Адрес локального Compose
Интерфейс	http://localhost:8080
Swagger	http://localhost:8080/api/docs
OpenAPI	http://localhost:8080/api/openapi.json
Готовность	http://localhost:8080/api/v1/ready
Mailpit	http://localhost:8025
API напрямую	http://localhost:8000
PostgreSQL с хоста	127.0.0.1:54329
SMTP Mailpit с хоста	127.0.0.1:1025

Порты привязаны к 127.0.0.1. Они доступны на компьютере запуска, но не автоматически другим участникам сети. Ответ ready должен быть HTTP 200; Swagger должен показывать методы текущей версии.

На рабочем компьютере подготовки документа прямой порт API 8000 занят другим сервисом. Для этого стенда применён локальный Compose override с портом 8010; интерфейс 8080 и Mailpit 8025 работают без изменения. Такой override относится к окружению разработчика и не является обязательным файлом проекта.

При аналогичном конфликте можно создать собственный override и заменить публикацию api. В Compose, поддерживающем тег !override, фрагмент имеет вид:

```
services:
  api:
    ports: !override
      - "127.0.0.1:8010:8000"
python3 scripts/dev.py -f docker-compose.yml \
  -f local-compose.override.yml up -d --build --wait
```

Регистрация и локальная почта

Открыть интерфейс, выбрать роль и зарегистрироваться с вымышленным адресом. Затем открыть Mailpit, найти письмо, перейти по ссылке и нажать подтверждение email. После подтверждения выполнить вход. Готовых общих паролей и обхода проверки email нет.

По умолчанию SMTP_HOST = mailpit, SMTP_PORT = 1025. Письмо появляется только в Mailpit; Gmail, Яндекс и другой внешний ящик его не получают. Для проверки на локальном стенде достаточно вымышленного адреса, поскольку письмо перехватывает Mailpit.

Для внешней доставки задать собственные SMTP_HOST, SMTP_PORT, SMTP_FROM, SMTP_USER, SMTP_PASSWORD и подходящий режим SMTP_STARTTLS либо SMTP_SSL. PUBLIC_APP_URL должен быть доступен получателю, иначе доставленная ссылка не откроет нужный стенд. После изменения конфигурации пересоздать API и проверить доставку.

Не следует публиковать SMTP_PASSWORD, сессионные cookie и ссылки подтверждения. Если SMTP недоступен, регистрация не превращается в автоматическое подтверждение: устранить причину и повторить отправку.

Разработка с горячей перезагрузкой

Сначала создать окружение и поднять только БД и Mailpit. Если полноценный Compose уже запущен, остановить web и api, чтобы исключить конфликт портов.

```
docker compose stop web api
python3.12 -m venv .venv
.venv/bin/python -m pip install \
  -r apps/api/requirements-dev.txt
docker compose up -d db mailpit
```

Для API открыть отдельный терминал:

```
cd apps/api
cp .env.example .env
```

В apps/api/.env задать DATABASE_URL для локального PostgreSQL, SMTP_HOST = localhost, SMTP_PORT = 1025 и PUBLIC_APP_URL = http://localhost:5173. Пример строки для демонстрационной БД:

```
DATABASE_URL=postgresql+psycopg://signal:signal-local-only@localhost:54329/
signal
```

Пароль signal-local-only относится только к исходному локальному примеру. При изменении POSTGRES_PASSWORD строка подключения должна содержать соответствующее локальное значение.

```
../../.venv/bin/alembic upgrade head
../../.venv/bin/uvicorn app.main:app --reload \
  --host 127.0.0.1 --port 8000 --no-access-log
```

Для интерфейса открыть другой терминал из корня проекта:

```
cd apps/web
npm ci
npm run dev
```

Открыть http://localhost:5173. Vite проксирует /api на локальный API. При выборе иного порта API изменить адрес прокси в конфигурации Vite. Пример API-конфигурации допускает SQLite для быстрого старта, но основной Compose и production используют PostgreSQL.

На Windows активировать .venv\Scripts\Activate.ps1 и вызывать python, alembic и uvicorn из активированного окружения. Вместо cp использовать Copy-Item. Параметры Docker Compose остаются теми же.

Проверки и обновление контрактов

```
.venv/bin/ruff check apps/api
.venv/bin/ruff format --check apps/api
PYTHONPATH=apps/api .venv/bin/pytest apps/api/tests -q
PYTHONPATH=apps/api .venv/bin/python \
  apps/api/scripts/export_contracts.py
cd apps/web
npm run api:types
npm run build
```

Браузерные проверки выполняются после готовности стенда 8080 и Mailpit 8025:

```
cd apps/web
npx playwright install chromium
npm run test:e2e
npm run test:e2e:d
```

По умолчанию API-тесты используют временный SQLite. Для PostgreSQL передать TEST_DATABASE_URL отдельной БД с именем, заканчивающимся _test. Набор очищает тестовые таблицы; передавать ему DATABASE_URL рабочего стенда нельзя.

Публичное развёртывание

Текущий Compose предназначен для локального стенда. Для сервера следует подготовить собственную конфигурацию, не меняя сохранённый рабочий volume. Внешне публикуется веб-вход; БД, SMTP-интерфейс и прямой API остаются во внутренней сети.

1. Подготовить сервер, домен и HTTPS-прокси перед Nginx приложения.
2. Создать отдельную PostgreSQL-базу, стойкий пароль и резервное копирование.
3. Передать секреты через защищённое окружение или секреты инфраструктуры.
4. Задать APP_ENV = production, COOKIE_SECURE = true и HTTPS PUBLIC_APP_URL.
5. Настроить реальный SMTP с TLS, проверить отправителя и письмо.
6. Собрать фиксированную версию, применить миграции и дождаться ready.
7. Проверить обе роли, письмо, тестирование, публикацию, подбор, принятие и отзыв контактов по внешнему адресу.
8. Зафиксировать версию, адрес прототипа, порядок резервирования и отката.

Production-конфигурация проверяется при старте: SQLite и SMTP без TLS не допускаются. Встроенный Mailpit не является внешним почтовым провайдером. Тексты политики обработки и согласий в MVP демонстрационные; перед использованием реальных персональных данных оператор утверждает полноценные документы, сроки хранения, порядок удаления и допустимую инфраструктуру в соответствии с ТЗ.

Для обновления сначала сделать резервную копию БД, затем собрать новую версию и применить миграции. При откате проверить совместимость схемы с прежним кодом; простой запуск старого образа не гарантирует обратимость изменений данных.

11 Сценарий демонстрации и дальнейшее развитие

Сквозной сценарий двух кабинетов

1. Зарегистрировать кандидата, подтвердить email через Mailpit и войти.
2. Заполнить профиль и контакты, принять обработку, сохранить. Скачать собственный PDF и проверить кириллицу.
3. Пройти опрос и тест, показать индивидуальные условия, черновик и серверный таймер. После успешной оценки подтвердить категорию.
4. Включить публикацию с отдельным согласием. Проверить выбранные публичные поля.
5. Показать сценарий без ФСП и, отдельно, связь собственного demo-ID. Подчеркнуть синтетический источник достижений.
6. В другом кабинете зарегистрировать работодателя, подтвердить email и создать компанию.
7. Создать потребность, открыть подбор, показать формулу, проверенные навыки, фильтры и сохранённую историю.
8. Направить кандидату адресное приглашение с описанием и зарплатой до налогов.
9. В кабинете кандидата открыть предложение, показать компанию, условия и историю. Принять с явным согласием на передачу контактов.
10. У работодателя проверить доступ к контактам. В кабинете кандидата отозвать доступ и показать, что повторное чтение больше не раскрывает контакты.
11. Скрыть профиль и повторно открыть сохранённый поиск работодателя: кандидат исключается из актуальной выдачи.

Для демонстрации обычное тестирование не обходится скрытой кнопкой или назначением грейда из браузера. Автоматический E2E использует контролируемые ответы для проверки программного пути; презентация людям должна показывать правила оценки и границы её подтверждённости.

Известные ограничения и последовательность развития

Следующий шаг — провести пилот с людьми и независимую разметку подбора, опубликовать обезличенные результаты и при необходимости выпустить новую версию заданий и порогов. Технические проверки не доказывают достоверность профессиональных грейдов.

После согласования с ФСП подключить официальный реестр и федеративную авторизацию, сохранив путь без истории. Далее расширять специализации, регулярные задания, модерацию и ATS. Вакансии и самостоятельные отклики дополняют основной путь адресного приглашения.

Для роста нагрузки измерить работу большого банка и заменить его полное чтение при ранжировании индексами и кешем, соблюдая отзыв приватности. Локальная нагрузочная серия не прогнозирует промышленную пропускную способность.

Перед сдачей добавить презентацию и прототип либо видео, опубликовать согласованные исходники и проверить ссылки. Проведение пилота, внешнее размещение и официальный реестр подтверждаются отдельно от настоящей документации.